

融合AI技术的校园社团活动报名管理小程序软件过程定义文档

文档信息

项目	内容
项目名称	校园社团活动报名管理小程序
文档版本	V2.0 (AI技术融合版)
适用标准	ISO/IEC 12207:2017、ISO/IEC 15504、ISO/IEC 33001
团队规模	5人
项目周期	2个月
技术栈	微信小程序原生框架 + Java后端

1 过程概述与目标

1.1 AI技术融合的背景与意义

随着大语言模型（LLM）、智能代码生成、自动化测试等人工智能技术的快速发展，软件工程领域正经历深刻变革。传统的软件过程管理模式虽然在标准化、可追溯性方面具备优势，但在小型团队项目中面临效率瓶颈——文档编写耗时、代码审查人力有限、测试覆盖难以全面等问题日益突出。

本项目“校园社团活动报名管理小程序”为典型的5人小团队敏捷项目，开发周期仅2个月。在此约束条件下，将AI技术有机融入已有的三层软件过程体系，具有以下重要意义：

1. 效率提升：利用AI辅助代码生成、自动化文档编写等能力，缩短单一任务的执行时间，使有限人力聚焦于高价值创造性工作。
2. 质量增强：借助AI进行代码静态分析、测试用例自动生成、缺陷模式识别，提升软件产品的内在质量属性。
3. 知识沉淀：AI工具可辅助完成需求文档结构化、设计文档模板填充、会议纪要自动整理等工作，降低知识流失风险。
4. 过程可度量性增强：引入AI贡献度指标，使过程改进有据可依，为后续项目提供量化参考。

本过程定义基于ISO/IEC 12207:2017中关于软件生命周期过程的框架要求，结合ISO/IEC 15504过程能力评估模型与ISO/IEC 33001过程评估概念框架，在原有三层过程剪裁结果的基础上进行AI增强改进。AI技术的引入不改变过程的基本结构与标准合规性，而是作为“过程增强手段”嵌入各活动环节。

1.2 新过程体系的整体目标

改进后的软件过程定义在保留原有基于三大国际标准剪裁所得的三层过程体系基础上，融入AI技术形成“AI增强型三层过程体系”，整体目标如下：

- 标准合规性：过程定义仍遵循ISO/IEC 12207生命周期过程框架，AI工具作为“过程增强手段”嵌入，不改变过程的基本结构与合规性要求。
- 人机协同：建立清晰的人-AI协作边界，AI承担辅助性、重复性工作，人类保留决策权与最终审批权，确保责任可追溯。
- 可度量改进：引入AI贡献度指标体系，实现过程绩效的量化评估与持续改进（PDCA循环）。
- 适度引入：根据项目规模与团队能力，选择性引入成熟AI工具，避免过度技术化带来的管理复杂度上升。

- 风险可控：AI生成物须经人工审查确认，建立AI产出的质量门控机制，确保技术风险可控。

1.3 过程体系结构概览

改进后的过程体系仍保持三层架构：

层级	过程类别	AI融合要点
顶层	核心生命周期基本过程（6个）	各阶段嵌入AI辅助活动节点
中层	项目支撑保障过程（3个）	AI辅助质量检查、进度预测、文档自动生成
底层	基础组织管理过程（2个）	AI工具协调员角色引入、风险智能预警

2 各阶段活动流程（含AI参与节点）

2.1 项目立项与需求调研过程

2.1.1 活动内容

本过程旨在明确项目目标、范围与核心需求，为后续设计开发奠定基础。主要活动包括：

1. 项目可行性分析：评估技术可行性、资源可行性与时间可行性。
2. 用户需求调研：通过问卷、访谈、竞品分析等方式收集目标用户（社团管理者、普通学生）的需求。
3. 需求文档编制：将调研结果结构化为软件需求规格说明书（SRS）。
4. 需求评审与确认：团队内部评审需求文档的完整性、一致性与可实现性。

2.1.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
竞品分析	ChatGPT/Claude	快速汇总市面同类产品功能对比，生成竞品分析矩阵
需求文档生成	AI写作助手	根据调研原始数据自动生成需求文档初稿，含用户故事模板填充
需求完整性检查	LLM提示词工程	输入需求列表，让AI检查是否存在遗漏的边界条件或异常场景
会议纪要整理	语音转文字+AI摘要	自动生成需求评审会议纪要

2.1.3 输入输出物

- 输入物：项目立项申请、用户调研原始数据、竞品资料
 - 输出物：项目可行性报告、软件需求规格说明书（SRS）、需求跟踪矩阵、竞品分析报告（AI辅助生成）
-

2.2 软件系统设计过程

2.2.1 活动内容

1. 架构设计：确定系统整体架构（前后端分离、微信小程序+Spring Boot后端+MySQL数据库）。
2. 数据库设计：设计ER模型与数据表结构。
3. 接口设计：定义前后端API接口规范。
4. UI/UX设计：完成小程序界面原型设计。
5. 设计评审：团队评审设计方案的合理性与可实现性。

2.2.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
架构方案建议	ChatGPT/Claude	输入功能需求与约束条件，获取架构设计建议与技术选型参考
数据库表结构生成	AI代码助手	根据实体关系描述自动生成DDL语句初稿
API接口文档生成	GitHub Copilot	根据功能描述生成RESTful API定义（OpenAPI格式）
设计文档补全	AI写作工具	自动填充设计文档模板中的标准化描述部分

2.2.3 输入输出物

- 输入物：软件需求规格说明书、技术约束条件
- 输出物：系统架构设计文档、数据库设计文档（含ER图）、API接口规范文档、UI原型稿、设计评审记录

2.3 软件编码开发实现过程

2.3.1 活动内容

1. 编码规范制定：确定代码风格、命名规范、Git分支管理策略。
2. 功能模块开发：按迭代计划逐模块实现功能代码。
3. 代码审查：团队成员互审代码质量。
4. 单元测试编写：为核心业务逻辑编写单元测试。

2.3.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
代码生成		

AI参与环节	工具/方法	具体用途
	GitHub Copilot / 通义灵码	根据注释或函数签名自动补全实现代码
代码审查辅助	AI Code Review工具	自动检测潜在Bug、代码异味、安全漏洞
单元测试生成	Copilot Chat	为已有方法自动生成JUnit/Mockito测试用例
代码重构建议	AI静态分析	识别重复代码、过长方法等问题并给出重构建议
注释与文档生成	AI Doc Generator	自动为类和方法生成Javadoc注释

2.3.3 输入输出物

- 输入物：系统设计文档、API接口规范、编码规范
- 输出物：源代码（含AI辅助生成标注）、单元测试代码、代码审查记录、编码阶段周报

2.4 软件分级测试验证过程

2.4.1 活动内容

1. 测试计划制定：明确测试范围、策略、资源分配与进度安排。
2. 测试用例设计：针对功能需求与非功能需求设计测试用例。
3. 测试执行：按计划执行单元测试、集成测试、系统测试。
4. 缺陷管理：记录、跟踪、验证缺陷修复情况。
5. 测试报告编制：汇总测试结果与质量评估。

2.4.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
测试用例自动生成	AI测试助手	

AI参与环节	工具/方法	具体用途
		根据需求文档与API定义自动生成测试用例集
缺陷模式分析	LLM分析	对历史缺陷数据进行模式识别，预测高风险模块
测试数据生成	AI数据生成器	自动构造边界值、等价类等测试数据
测试报告生成	AI文档工具	根据测试执行结果自动生成测试报告初稿
回归测试优先级排序	ML模型	基于代码变更范围智能推荐回归测试优先执行集

2.4.3 输入输出物

- 输入物：需求文档、设计文档、源代码、测试计划
- 输出物：测试用例集（含AI生成标注）、测试执行记录、缺陷报告、测试总结报告

2.5 软件部署上线交付过程

2.5.1 活动内容

1. 部署环境准备：配置服务器环境、数据库初始化、域名与SSL证书配置。
2. 部署脚本编写：编写自动化部署脚本（CI/CD流水线）。
3. 上线前检查：执行预发布环境验证。
4. 正式上线：完成小程序审核提交与后端服务部署。
5. 上线后监控：监控系统运行状态与用户反馈。

2.5.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
部署脚本生成	AI代码助手	根据部署需求自动生成Docker Compose/CI配置文件
配置检查	AI审查工具	自动检查配置文件中的安全隐患（如硬编码密钥、开放端口）
运维文档生成	AI写作工具	自动生成部署手册与运维操作指南
异常日志分析	LLM日志分析	对上线后的异常日志进行智能分类与根因建议

2.5.3 输入输出物

- 输入物：通过测试的软件包、部署环境需求说明
- 输出物：部署脚本、部署手册、上线检查清单、运行监控报告

2.6 项目验收与交付归档过程

2.6.1 活动内容

1. 验收测试：按照验收标准执行最终确认测试。
2. 项目总结：编写项目总结报告，归纳经验教训。
3. 文档归档：整理并归档全部项目文档。
4. 知识转移：完成运维交接与知识转移。

2.6.2 AI工具使用方式

AI参与环节	工具/方法	具体用途
项目总结报告生成	AI写作助手	

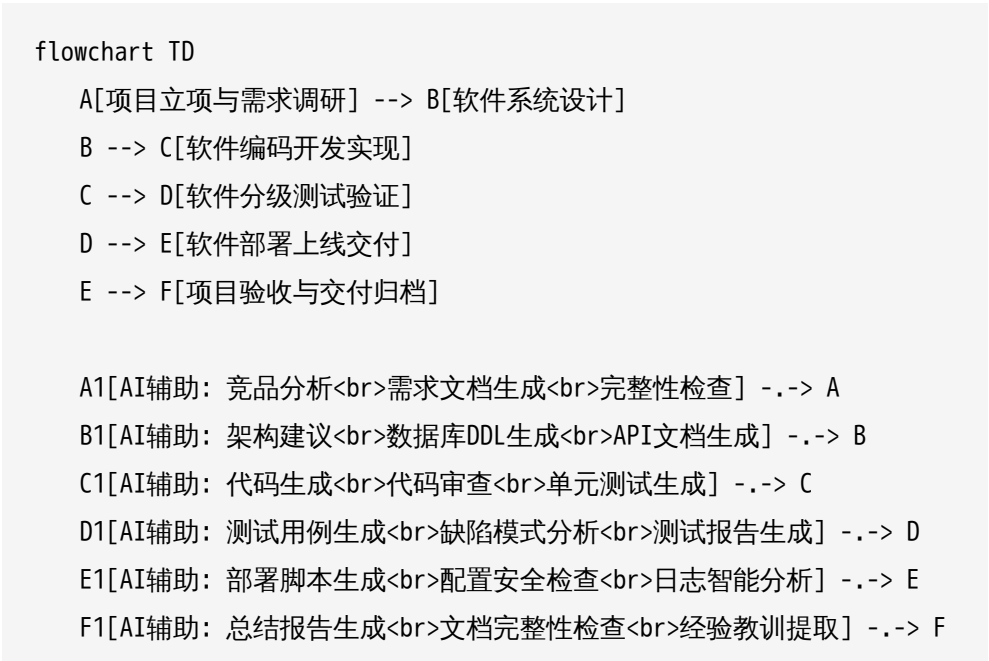
AI参与环节	工具/方法	具体用途
		根据各阶段记录自动生成项目总结报告框架
文档完整性检查	AI检查工具	扫描文档清单，检查是否存在缺失的必要文档
经验教训提取	LLM分析	从项目过程数据中提取关键经验教训条目
知识库构建	AI知识管理	将项目产出结构化存入团队知识库供后续复用

2.6.3 输入输出物

- 输入物：全部阶段性交付物、项目过程记录
- 输出物：验收测试报告、项目总结报告、归档文档包、经验教训库

2.7 核心流程图

流程图1：顶层生命周期过程与AI参与节点总览



```
style A1 fill:#e1f5fe
style B1 fill:#e1f5fe
style C1 fill:#e1f5fe
style D1 fill:#e1f5fe
style E1 fill:#e1f5fe
style F1 fill:#e1f5fe
```

流程图2：软件编码开发实现过程的AI协作详细流程

```
flowchart LR
    Start([开始编码]) --> A[领取开发任务]
    A --> B{是否有AI可辅助环节?}
    B -->|是| C[AI生成代码初稿]
    B -->|否| D[人工编写代码]
    C --> E[开发者审查AI代码]
    E --> F{AI代码质量是否达标?}
    F -->|是| G[采纳并适配修改]
    F -->|否| H[人工重写/修正]
    G --> I[提交代码]
    H --> I
    D --> I
    I --> J[AI辅助代码审查]
    J --> K{是否发现问题?}
    K -->|是| L[开发者修复问题]
    L --> I
    K -->|否| M[AI生成单元测试]
    M --> N[执行测试]
    N --> O{测试通过?}
    O -->|是| P[合并至主分支]
    O -->|否| Q[定位并修复缺陷]
    Q --> N
    P --> End([编码完成])
```

```
style C fill:#e8f5e9
style J fill:#e8f5e9
style M fill:#e8f5e9
```

流程图3：软件分级测试验证过程的AI协作流程



3 中层项目支撑保障过程（AI增强）

3.1 质量管控过程

在传统的代码评审、文档检查基础上，引入AI质量辅助手段：

- AI代码质量扫描：每次代码提交触发AI自动审查，检测代码异味、潜在缺陷、安全漏洞，生成质量报告。
- AI文档质量评估：利用AI工具检查文档的完整性、一致性、术语规范性，输出改进建议。
- 质量趋势预测：基于历史缺陷数据，AI预测后续迭代可能的质量风险区域。

- 质量门控机制：设置AI质量检查作为代码合入主分支的必要条件之一，未通过AI扫描的代码不可合入。

3.2 进度与成本管控过程

- AI进度预测：根据已完成任务的实际耗时与剩余任务复杂度，AI估算项目进度偏差并预警。
- 工时自动统计：AI辅助分析Git提交频率、代码量变化等客观数据，辅助工时评估。
- 资源瓶颈识别：AI分析任务分配与完成情况，识别团队中可能存在的资源瓶颈。
- 燃尽图自动更新：AI根据任务完成状态自动更新项目燃尽图，可视化进度偏差。

3.3 文档配套编制管理过程

- 文档模板智能填充：AI根据项目上下文自动填充标准文档模板中的通用部分。
 - 文档版本对比：AI辅助生成文档版本差异摘要，便于评审。
 - 文档关联性检查：AI检查各文档间的引用关系是否正确、是否存在不一致。
 - 文档标准化检查：AI审查文档格式、术语、编号等是否符合团队规范。
-

4 底层基础组织管理过程（AI增强）

4.1 团队人员岗位职责管理

引入AI工具协调员角色后，需要对团队职责进行重新定义（详见第5章）。AI工具的引入不替代任何人员岗位，而是作为辅助手段增强各岗位的工作效能。核心原则：

- 人为主、AI为辅：所有AI产出须经人工审查确认，决策权归于人类。
- 职责边界清晰：每个AI辅助环节均有明确的人类负责人。

- 能力提升导向：AI工具使用旨在提升团队能力，而非替代人力。

4.2 项目风险简易管控

- AI风险识别：AI根据项目计划、进度偏差、缺陷趋势等数据自动识别风险信号。
- 风险应对建议：AI基于风险类型推荐应对策略（如增加测试投入、调整排期等）。
- 风险周报自动生成：AI汇总本周风险状态变化，自动生成风险周报初稿。
- AI工具自身风险管理：对AI工具可能引入的风险（如生成代码质量不稳定、知识产权问题等）进行专项管控。

5 角色职责定义

5.1 角色总览

角色	人员	兼任AI角色	主要AI工具
项目经理	1人	AI工具协调员	ChatGPT、项目管理AI助手
前端开发工程师	1人	—	GitHub Copilot、通义灵码
后端开发工程师	1人	—	GitHub Copilot、通义灵码
测试工程师	1人	AI测试分析师	AI测试助手、LLM分析工具
运维工程师	1人	—	AI代码助手、日志分析工具

5.2 各角色职责清单

5.2.1 项目经理

基本职责： - 制定项目计划与里程碑，管理项目进度与成本 - 组织需求评审、设计评审等关键评审活动 - 协调团队沟通，解决项目执行中的障碍 - 编制项目状态报告，向利益相关方汇报 - 管理项目风险与变更

兼任AI工具协调员职责： - 评估与选型适合本项目的AI工具集，建立AI工具清单 - 制定AI工具使用规范与审批流程 - 监控AI工具使用效果，收集AI贡献度数据 - 组织AI工具使用培训，确保团队成员正确使用 - 评估AI生成物的合规性与知识产权风险 - 定期评审AI工具使用策略并提出改进建议 - 维护AI贡献度度量数据，纳入项目周报

5.2.2 前端开发工程师

基本职责： - 负责微信小程序前端界面开发与交互实现 - 与UI设计稿保持高度一致的页面还原 - 与后端工程师协作完成接口对接 - 编写前端单元测试 - 参与代码审查

AI协作职责： - 使用AI代码助手辅助完成页面组件开发 - 对AI生成的前端代码进行质量审查与适配修改 - 标注代码中AI辅助生成的部分（使用约定注释格式） - 反馈AI工具在前端开发中的适用性问题 - 记录AI代码采纳率与修改率数据

5.2.3 后端开发工程师

基本职责： - 负责Java后端服务的设计与开发 - 数据库设计与优化 - API接口实现与文档维护 - 编写后端单元测试与集成测试 - 参与代码审查

AI协作职责： - 使用AI代码助手辅助完成业务逻辑与数据访问层代码 - 利用AI工具生成API接口文档与数据库DDL - 对AI生成的后端代码进行安全性审查 - 标注代码中AI辅助生成的部分（使用约定注释格式） - 记录AI代码采纳率与修改率数据

5.2.4 测试工程师

基本职责： - 制定测试计划与测试策略 - 设计与编写测试用例 - 执行功能测试、接口测试、性能测试 - 管理缺陷生命周期 - 编写测试报告

兼任AI测试分析师职责： - 利用AI工具自动生成测试用例并进行有效性筛选 - 使用AI进行缺陷模式分析与高风险模块预测 - 评估AI生成测试用例的覆盖率与有效性 - 维护AI测试工具的提示词模板与配置 - 对比AI测

试用例与人工测试用例的缺陷发现能力 - 输出AI测试贡献度分析报告 - 记录AI测试用例有效率数据

5.2.5 运维工程师

基本职责： - 搭建与维护开发、测试、生产环境 - 编写自动化部署脚本与CI/CD流水线 - 监控系统运行状态与性能指标 - 处理线上故障与应急响应 - 编写运维文档与操作手册

AI协作职责： - 使用AI辅助生成部署配置文件与运维脚本 - 利用AI进行日志智能分析与异常根因定位 - 对AI生成的配置文件进行安全性审查 - 反馈AI工具在运维场景中的适用性 - 记录AI辅助运维的效率提升数据

6 交付物清单与质量标准

6.1 各阶段交付物清单

阶段	交付物	AI参与标注	质量负责人
立项与需求调研	项目可行性报告	—	项目经理
	软件需求规格说明书（SRS）	AI辅助生成初稿	项目经理
	竞品分析报告	AI辅助汇总	项目经理
	需求跟踪矩阵	—	项目经理
系统设计	系统架构设计文档	AI提供建议参考	后端开发
	数据库设计文档	AI生成DDL初稿	后端开发
	API接口规范文档	AI辅助生成	后端开发
	UI原型设计稿	—	前端开发
编码开发	源代码（前端+后端）	AI辅助生成部分代码	前端/后端开发

阶段	交付物	AI参与标注	质量负责人
	单元测试代码	AI辅助生成	前端/后端开发
	代码审查记录	含AI审查报告	全体开发
测试验证	测试计划	—	测试工程师
	测试用例集	AI辅助生成部分用例	测试工程师
	缺陷报告	AI辅助分类分析	测试工程师
	测试总结报告	AI辅助生成初稿	测试工程师
部署上线	部署脚本与CI/CD配置	AI辅助生成	运维工程师
	部署手册	AI辅助生成	运维工程师
	上线检查清单	—	运维工程师
验收归档	验收测试报告	—	测试工程师
	项目总结报告	AI辅助生成框架	项目经理
	归档文档包	AI完整性检查	项目经理
	AI贡献度分析报告	AI数据汇总	项目经理

6.2 质量验收标准

6.2.1 文档类交付物质量标准

质量维度	验收标准
完整性	文档包含模板规定的全部章节，无遗漏项
一致性	文档间引用关系正确，术语统一，数据一致
准确性	技术描述准确无误，与实际实现一致
可追溯性	需求可追溯至设计、代码与测试用例
AI标注规范	AI辅助生成的内容须明确标注，并经人工审查确认

6.2.2 代码类交付物质量标准

质量维度	验收标准
功能正确性	通过全部单元测试与集成测试
代码规范	符合团队编码规范，无严重代码异味
测试覆盖率	核心业务逻辑单元测试覆盖率≥70%
安全性	无高危安全漏洞（经AI安全扫描+人工审查）
AI代码审查	AI生成代码须经开发者逐行审查并标注确认
可维护性	代码注释完整，AI生成代码可读性达到团队标准

6.2.3 AI生成物专项质量标准

针对AI辅助生成的交付物，额外适用以下质量要求：

- 1. 人工审查原则：所有AI生成的内容必须经过对应负责人的人工审查与确认，未经审查的AI产出不得直接纳入正式交付物。
 - 2. 来源标注原则：AI辅助生成的代码段、文档段落须通过注释或标记明确标注为”AI辅助生成”，格式示例：
 - 代码：`// [AI-Generated] by Copilot, reviewed by <developer_name>`
 - 文档：[本节由AI辅助生成，经<负责人>审查确认]
 - 3. 正确性验证原则：AI生成的技术内容（代码、配置、SQL等）须经功能验证确认其正确性。
 - 4. 一致性原则：AI生成内容须与项目整体风格、术语、架构设计保持一致。
 - 5. 知识产权合规原则：确保AI生成的代码不存在版权风险，不包含敏感或受限代码片段。
-

7 过程度量指标（含AI贡献度指标）

7.1 传统过程度量指标

指标名称	计算公式	目标值	采集频率
进度偏差率 (SV%)	$(\text{实际完成} - \text{计划完成}) / \text{计划完成} \times 100\%$	$\leq \pm 10\%$	每周
需求变更率	$\text{变更需求数} / \text{总需求数} \times 100\%$	$\leq 15\%$	每迭代
缺陷密度	$\text{缺陷总数} / \text{代码千行数}$	$\leq 5 \text{个/KLOC}$	测试阶段
缺陷修复率	$\text{已修复缺陷数} / \text{发现缺陷总数} \times 100\%$	$\geq 95\%$	每周
代码覆盖率	$\text{已覆盖代码行数} / \text{总代码行数} \times 100\%$	$\geq 70\%$ (核心模块)	每次提交
测试通过率	$\text{通过用例数} / \text{总用例数} \times 100\%$	$\geq 95\%$	每轮测试
文档完成率	$\text{已完成文档数} / \text{应交付文档数} \times 100\%$	100%	每阶段
返工率	$\text{返工任务数} / \text{总任务数} \times 100\%$	$\leq 10\%$	每迭代

7.2 AI贡献度指标

指标名称	计算公式	说明	采集频率
AI代码采纳率	$\text{AI生成代码被采纳行数} / \text{AI生成代码总行数} \times 100\%$	反映AI代码生成的实际可用性	每周
AI代码占比			每迭代

指标名称	计算公式	说明	采集频率
	AI辅助生成代码行数 / 总代码行数 × 100%	反映AI在编码工作中的参与程度	
AI测试用例有效率	AI生成且被纳入执行的用例数 / AI生成总用例数 × 100%	反映AI测试用例的质量	每轮测试
AI测试缺陷发现率	AI生成用例发现的缺陷数 / 总发现缺陷数 × 100%	反映AI测试用例的缺陷发现能力	每轮测试
AI辅助文档完成度	AI辅助生成的文档章节数 / 总文档章节数 × 100%	反映AI在文档工作中的辅助程度	每阶段
AI代码审查问题发现率	AI审查发现的问题数 / 总发现问题数 × 100%	反映AI代码审查的有效性	每周
AI建议采纳率	采纳的AI建议数 / AI总建议数 × 100%	反映AI建议的整体质量	每周
AI工具使用频率	团队成员平均每日使用AI工具次数	反映AI工具的使用活跃度	每日

7.3 度量数据采集与分析

7.3.1 数据采集方式

数据来源	采集方式	负责人
Git提交记录	自动化脚本统计AI标注代码行数	运维工程师
AI工具使用日志	AI工具内置统计功能导出	AI工具协调员
测试管理工具	从测试用例管理系统导出统计	AI测试分析师
代码审查平台	统计AI审查报告中的问题数	后端开发
项目管理工具	任务完成情况与进度数据	项目经理

7.3.2 度量分析与改进

- 周度分析：每周项目例会中回顾AI贡献度指标，识别AI工具使用中的问题。
 - 迭代回顾：每个迭代结束时，对比AI贡献度指标与传统指标，评估AI工具的实际效果。
 - 过程改进：基于度量数据识别改进机会，调整AI工具使用策略（如更换效果不佳的工具、优化提示词等）。
 - 最终评估：项目结束时输出完整的AI贡献度分析报告，为后续项目提供参考基线。
-

8 AI工具使用规范与治理

8.1 AI工具使用基本原则

1. 辅助而非替代：AI工具定位为团队的辅助手段，不替代人类的判断与决策。
2. 审查优先：所有AI产出在纳入正式交付物之前，必须经过人工审查。
3. 标注透明：AI参与生成的内容必须明确标注，确保过程可追溯。
4. 数据安全：禁止将项目敏感信息（如用户隐私数据、系统密钥等）输入AI工具。
5. 合规使用：遵守AI工具的使用协议，确保知识产权合规。

8.2 AI工具审批流程

引入新AI工具须经过以下审批流程：

1. AI工具协调员（项目经理）评估工具的功能、安全性与适用性。
2. 与团队讨论确认引入必要性与使用范围。
3. 制定该工具的具体使用规范（适用场景、禁用场景、数据安全要求）。
4. 在小范围试用后推广至全团队。

8.3 AI生成物的知识产权管理

- 使用AI工具前确认其生成物的知识产权归属条款。
 - AI生成的代码在合入主分支前须经人工审查，确认不包含可识别的第三方受版权保护代码。
 - 项目文档中标注AI辅助生成的部分，在必要时可提供人工修改记录。
-

9 过程持续改进机制

9.1 PDCA循环在AI增强过程中的应用

阶段	活动	AI辅助内容
Plan（计划）	制定AI工具使用计划与目标值	AI分析历史数据提供基线参考
Do（执行）	按计划在各过程环节使用AI工具	AI参与各阶段具体活动
Check（检查）	收集度量数据，评估AI效果	AI汇总分析度量数据
Act（改进）	调整策略，优化工具使用方式	AI提供改进建议

9.2 迭代回顾与改进

每个迭代结束时召开回顾会议，专题讨论： - AI工具使用中遇到的问题与解决方案 - AI贡献度指标达成情况与偏差原因 - 下一代AI工具使用策略调整

9.3 知识积累与共享

- 建立团队AI工具使用经验库（优秀提示词、最佳实践、踩坑记录）。
 - 每位团队成员定期分享AI工具使用心得。
 - 项目结束时将AI使用经验纳入组织过程资产库。
-

10 总结

本文档在原有基于ISO/IEC 12207、ISO/IEC 15504、ISO/IEC 33001三大标准剪裁所得的三层软件过程体系基础上，系统性地融入了AI技术，形成了“AI增强型三层过程体系”。改进后的过程定义具有以下特征：

1. 结构完整：保留了原有三层过程架构的完整性，AI技术作为增强手段嵌入而非重构。
2. 标准合规：过程定义仍满足ISO/IEC 12207生命周期过程框架的基本要求。
3. 人机协同：建立了清晰的人-AI协作边界与审批机制，确保人类对AI产出保持控制权。
4. 可度量：引入了AI贡献度指标体系，使AI技术的效果可量化评估。
5. 可持续改进：通过PDCA循环与迭代回顾机制，支持AI使用策略的持续优化。

该过程定义适用于本项目（校园社团活动报名管理小程序）的全生命周期管理，为5人团队在2个月开发周期内高效、高质量地交付软件产品提供了过程保障与规范指引。

文档结束